shavar

Seven implementations of SHA-256, written as a two-dimensional recurrence

What each language makes natural, and what it makes hard

9 August 2026

Abstract

SHA-256 is normally presented as eight working registers that shuffle on every round. Six of those eight assignments are pure copies. Removing them leaves two coupled recurrences with a lookback of four, which is both smaller to write and closer to the object that cryptanalysis actually reasons about. This note reports on implementing that formulation seven times — in C99, Lean 4, Python, Perl, Scheme, JavaScript and shell — under a rule of *core language only, no libraries*, and on what the exercise revealed about the languages rather than about the algorithm. All seven agree byte-for-byte on every intermediate value, which is the evidence that the comparison is between seven correct programs rather than seven different bugs.

Contents

1 The reformulation

FIPS 180-4 [3] defines the compression function over eight variables $a \dots h$, updated each round as

$$T_1 = h \boxplus \Sigma_1(e) \boxplus \mathrm{Ch}(e,f,g) \boxplus K_t \boxplus W_t, \qquad T_2 = \Sigma_0(a) \boxplus \mathrm{Maj}(a,b,c),$$
$$h \leftarrow g, \quad g \leftarrow f, \quad f \leftarrow e, \quad e \leftarrow d \boxplus T_1, \qquad d \leftarrow c, \quad c \leftarrow b, \quad b \leftarrow a, \quad a \leftarrow T_1 \boxplus T_2,$$

where $\boxplus$ is addition modulo 2^{32}. Six of those eight assignments move a value without changing it. They are not computation; they are a shift register written out longhand.

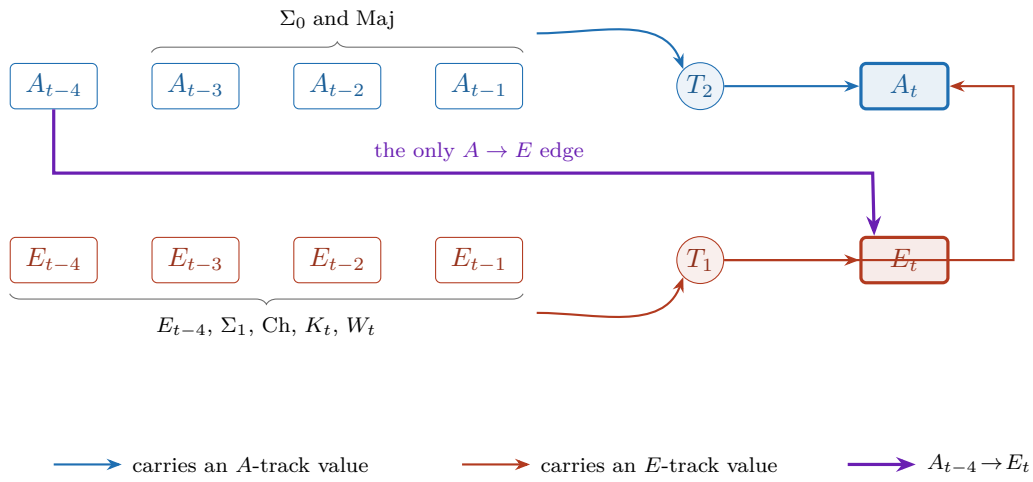

Figure 1: One round of the recurrence of Equation (1). A box is a 32-bit word; a circle is an accumulator; an arrow is a word flowing into an arithmetic step. Blue arrows read the A track, red arrows read the E track, and the single violet edge $A_{t-4} \to E_t$ is the one place where the A track feeds the E track. Advancing a round shifts both rows one cell to the left; nothing is copied.

Name the two computed sequences and index them by round. With $A_{-1}=H_0, A_{-2}=H_1, A_{-3}=H_2, A_{-4}=H_3$ and $E_{-1}=H_4, \ldots, E_{-4}=H_7$, the whole compression is

$$\boxed{\begin{aligned} T_1[t] &= E_{t-4} \boxplus \Sigma_1(E_{t-1}) \boxplus \mathrm{Ch}(E_{t-1}, E_{t-2}, E_{t-3}) \boxplus K_t \boxplus W_t \\ T_2[t] &= \Sigma_0(A_{t-1}) \boxplus \mathrm{Maj}(A_{t-1}, A_{t-2}, A_{t-3}) \\ E_t &= A_{t-4} \boxplus T_1[t] \\ A_t &= T_1[t] \boxplus T_2[t] \end{aligned}} \tag{1}$$

The correspondence with the standard form is exactly $a = A_{t-1}$, $b = A_{t-2}$, $c = A_{t-3}$, $d = A_{t-4}$ and $e = E_{t-1}, \ldots, h = E_{t-4}$: the eight registers are two sliding windows of width four. Figure 1 draws one round.

This is not a new observation — it is how the function is handled in the cryptanalytic literature — but it is rarely how it is *implemented*, and implementing it this way turns out to change the character of the code in every language. Section 3 is mostly about that.

Two properties worth having in view

The state *is* the history.
Keeping all of $A_{-4..63}$ and $E_{-4..63}$ costs 136 words per block. Every intermediate value is then addressable after the fact at no extra cost, which is what makes the code usable as an instrument rather than only as a hash.

The tracks are not symmetric.
A_t reads the E history at three points (through Ch) plus E_{t-4}; but E_t reads the A history at exactly one point, A_{t-4}. That single edge is the only path by which the A track influences the E track at all. It is drawn thick and violet in Figure 1 because it is easy to miss in the algebra and hard to miss in a picture.

2 The seven implementations at a glance

Every implementation obeys the same command-line contract, so any one can be diffed against any other not merely on the final digest but on the full per-round interior: W, A, E, T_1, T_2 for all

64 rounds. That is what *agree* means in Table 1 — 344 identical lines of trace, not 64 identical hex characters.

Language	Runtime	Lines	ms/block	Character of the port
C99	clang 17	315	0.0005	the reference; zero allocations
Lean 4	4.32.2	1851	—	implementation *and* proofs
Python	3.9.6	776	0.63	operator overloading, generators
Perl	5.34.1	539	0.16	`vec()`, `pack`, negative indices
Scheme	chibi 0.12 (R7RS)	1106	6.6	`syntax-rules`; no bitwise ops exist
JavaScript	JavaScriptCore	787	0.017	`Uint32Array`; no integer type
shell	bash 3.2 / zsh 5.9	811	14.9 / 2.4	builtins only; no subshells

Table 1: Line counts are the implementation file only, excluding tests and excluding the Lean proofs' share. Timings are per 512-bit block on an M-series Mac with other work running concurrently; they are honest to about a factor of two and are included for their *ratios*, not their absolute values. C is roughly 3×10^4 times faster than bash.

The line counts deserve a caveat before anyone reads them as a brevity ranking. Every file here is heavily commented for a reader who does not know the language, which was a project requirement; comment density varies more than code density does. C is the shortest partly because it is expressive for this task and partly because `uint32_t` does for free what four other languages spend thirty lines simulating.

3 Where the language helps, and where it fights

3.1 The one problem every language except two has

SHA-256 is defined on 32-bit words with wraparound. Exactly two of the seven languages have that type natively: C (`uint32_t`) and Lean (`BitVec 32`). The other five must simulate it, and *how* they simulate it is the single largest stylistic difference between the ports.

- **Python** confines the mask to one place by giving a `Word32` class overloaded arithmetic. The round body then reads like Equation (1) instead of like the equation plus fourteen `& 0xFFFFFFFF`. A genuine payoff: `~x` on a Python integer is negative, and masking inside the constructor turns it into the correct 32-bit complement for free.
- **JavaScript** has no integer type at all. Bitwise operators coerce to *signed* `int32`, so `0xFFFFFFFF | 0` is -1 and the natural big-endian assembly goes negative on any high byte. `>>> 0` is the only unsigned-producing operator. The clean fix is `Uint32Array`, which is core language, not a library: every store passes through `ToUint32`, giving real machine-word wraparound.
- **Perl** and **shell** mask by hand at every site. Both are 64-bit signed underneath, so a missed mask is a silently wrong digest far downstream with no type to catch it.
- **Scheme** has the same problem plus a larger one, below.

3.2 R7RS-small has no bitwise operations

Not "no rotate" — no `and`, `or`, `xor`, or shift, at all. The base library of R7RS [5] simply does not include them. A portable Scheme implementation of a function *defined* in terms of those operations must therefore rebuild the entire layer from `quotient` and `remainder`. This is the single largest structural cost imposed by any language in the set: roughly forty lines and an eightfold slowdown before the algorithm proper begins. The port handles it with `cond-expand` to pick up host primitives where they exist, and *tests both paths*, since a fallback that is never executed is a fallback that does not work.

3.3 The lookback window: four ways to say A_{t-4}

Equation (1) indexes backwards from the current round. Each language expresses that differently, and the differences are instructive:

Language	How A_{t-4} is written
Python, Perl	`A[-4]`, literally. If the list holds exactly the history so far, the language's own negative indexing *is* the lookback window and no index arithmetic appears anywhere. This is the closest any port gets to the equation. It carries a trap: the append order matters (`E.append` must precede `A.append`), and once the list stops growing, `A[-4]` silently means A_{60} instead. Python's port needs a separate accessor class for the finished trace for exactly this reason — the best feature in the file is also the most dangerous.
C, JavaScript	`A[4 + t]`, with the array offset by four so that t may be negative. Explicit, checkable, and needs a comment warning about the off-by-one.
Scheme	A `syntax-rules` macro, so the four lines of Equation (1) are the literal source text with the indexing pushed into the macro. This is the only port where the spec and the code are the same characters. Macro hygiene makes it safe in a way a C preprocessor macro would not be.
Lean	A `Vector` whose length is part of its type, so an out-of-range index is not a runtime error but a program that does not compile.
shell	Indexed arrays and a global out-parameter, because shell functions return only an exit status. There is no abstraction available to hide it behind.

3.4 Three findings that surprised us

JavaScript cannot return an exit code. The language has no notion of a process. Under `jsc`, Safari's engine, `quit()` *ignores its argument and always exits 0*; the only nonzero status it ever produces is 3, for an uncaught exception. A three-valued exit contract that every other language gets for free therefore required making the CLI a shell/JavaScript polyglot whose first two lines re-invoke the engine and translate a sentinel. This is a property of the host, not of the language, but it is exactly the kind of thing a portability comparison exists to surface.

Shell is not one data point. The same source runs about *four to six times faster under zsh 5.9 than under bash 3.2.57*. Meanwhile the micro-optimisation one would expect to matter — collapsing each round into a single comma expression rather than a dozen `(( ))` statements — is worth nothing measurable in either shell. Shell arithmetic cost lives in operators and variable lookups, not statement dispatch. Both facts were measured after being predicted wrongly.

An alias broke a proof tactic, silently. In Lean, `abbrev Word := BitVec 32` — fully reducible and apparently harmless — makes `bv_decide` fail. The alias survives inside instance arguments of the elaborated term, the bit-blaster matches those syntactically, and the tactic then reports a *spurious counterexample* rather than an error. Failing "successfully" is the worst possible failure mode for a verification tool, and the fix was to delete the alias and write `BitVec 32` everywhere.

4 What is proved, and what is merely tested

These are different kinds of evidence and the distinction is worth keeping sharp. Table 3 states which is which.

	Claim	Evidence	Status
V1	2D form ≡ 8-register form	Lean, by induction over rounds; lifted to block and whole-hash level	proved, kernel-only
V2	Bitwise identities (Ch, Maj, Σ, σ)	31 theorems; 11 kernel-only, 20 via `bv_decide`	proved
V3	Padding lands on a 512-bit multiple	Lean, arithmetic on L mod 512	proved, kernel-only
V4	Padding is injective	Lean, given well-formedness	proved, kernel-only
V5	Agreement with FIPS 180-4	Known-answer vectors	tested
V6	The seven agree with each other	Full per-round traces, swept bit lengths	tested

Table 3: There is no `sorry` anywhere in `lean/`. V1's proof is *kernel-only*: `#print axioms` reports `[propext, Quot.sound]` and nothing else.

On trusting `bv_decide`. The tactic bit-blasts a goal to CNF, calls a bundled CaDiCaL, and checks the returned LRAT certificate inside Lean [2]. The SAT solver is *not* trusted. The certificate check runs as compiled native code, so the Lean compiler is. On this toolchain that shows up as a per-theorem axiom `<name>._native.bv_decide.ax_N`, which `#print axioms` makes visible — and which the build prints on every run rather than asserting cleanliness in prose. Notably, the two identities that matter most are proved *twice*, once by `bv_decide` and once by case analysis on bit positions, and the second route needs no compiler trust at all.

An honest gap. V3 and V4 are proved about an abstract bit-level padding function, and a separate lemma links the concrete byte-level padder's *length* to it. What is *not* proved is that the byte padder emits the same bit string as the abstract one, only that it emits one of the right length. That gap is real and is marked as such in the source.

Why V5 and V6 do not subsume one another. V5 catches a shared misreading of the standard; V6 catches a transcription slip in one language — and because it compares traces, it names the round where the slip happened. Seven implementations could agree with each other and all be wrong, or all match NIST on byte-aligned input and diverge at $L = 5$.

The limit of external validation. OpenSSL, LibreSSL, `shasum` and `sha256sum` all hash whole bytes only. *No external oracle on a normal machine can validate a message whose length is not a multiple of eight.* For those cases the evidence is cross-implementation agreement plus NIST's bit-oriented CAVP vectors [4], and the repository says so rather than letting seven-way agreement stand in for external validation.

Prior work is worth naming here for scale: Appel's machine-checked proof of the OpenSSL C implementation of SHA-256 [1] verifies a real optimised C program against a functional specification in Coq, which is a substantially harder undertaking than anything in this repository. What `lean/` proves is the correctness of a *reformulation*, not of an optimised implementation.

5 Using it as an instrument

The reason for keeping the whole trajectory (§1) is that the intended reader is studying the function, not hashing a file. Every implementation therefore also offers the compression function on a caller-supplied chaining value and for a reduced number of rounds — neither reachable through a normal hashing API, and both prerequisites for free-start and reduced-round analysis.

Two structural facts the formulation makes visible, and which the code keeps separately addressable rather than fusing into one expression:

The linear/nonlinear split.
$\oplus$, rotation and shift are GF(2)-linear, hence so are $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$. The nonlinearity lives entirely in Ch, Maj, and the carries of $\boxplus$. Replace those and the function collapses to an affine map over $\mathrm{GF}(2)^{256}$.

Bit-slices are coupled only by carries.
Ch, Maj and $\oplus$ act bit-position-wise; rotations permute positions without mixing them. The *only* operation moving information from bit i to bit $j > i$ is the carry in $\boxplus$ — and it moves one way, towards the most significant bit. Replace every $\boxplus$ with $\oplus$ and SHA-256 decomposes into 32 independent one-bit ciphers. This asymmetry is why low-order differences are cheap to control and high-order ones are not.

The browser page in `js/index.html` exists to make this visible rather than merely stated: it steps through rounds forwards and backwards, draws the order-4 window and the single $A \to E$ edge of Figure 1 on live data, renders words as individual bits, and in diff mode shows a one-bit input difference avalanching across 64 rounds as a Hamming-weight curve.

6 Conclusions

1. **The reformulation is worth it.** In every language the round body became four lines with no permutation to get wrong, and the six copy assignments of the standard form vanished entirely. In Lean the equivalence to the standard form is *definitional* for a single round — which is the formal statement of "the register shuffle was never computation".

2. **The decisive language feature was not what we expected.** It was not macros, or pattern matching, or types. It was whether the language has a 32-bit unsigned integer. Two do; the other five spend their idiom budget simulating one, and that choice — wrapper class, typed array, or manual masking — shapes the rest of the file more than anything else.

3. **Negative indexing is the sleeper feature.** Python and Perl transcribe the recurrence with no index arithmetic at all, because a list's negative indices already *are* a lookback window. It is also the most dangerous feature in both files.

4. **Seven-way trace agreement is a strong instrument and a weak oracle.** It localises a disagreement to a round and a register, which no digest comparison can do. It cannot tell you all seven are not wrong together, which is what the NIST vectors are for.

References

[1] Andrew W. Appel. "Verification of a Cryptographic Primitive: SHA-256". In: *ACM Transactions on Programming Languages and Systems* 37.2 (2015). Local copy: `refs/appel-verif-sha256.pdf`, retrieved 2026-08-09 from the author's page at Princeton., 7:1–7:31. DOI: 10.1145/2701415. URL: https://www.cs.princeton.edu/~appel/papers/verif-sha.pdf (cit. on p. 5).

[2] Henrik Böving et al. "Interactive Bitvector Reasoning using Verified Bit-Blasting". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025). **No local copy**: the ACM Digital Library version could not be retrieved automatically on 2026-08-09. Bibliographic details verified against the ACM DL listing; the paper describes `bv_decide`, the tactic used throughout `lean/`., pp. 3259–3285. DOI: 10.1145/3763167 (cit. on p. 5).

[3] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Tech. rep. FIPS PUB 180-4. Local copy: `refs/fips180-4.pdf`, retrieved 2026-08-09 from nvlpubs.nist.gov. Identity confirmed from the PDF's own XMP metadata (*Secure Hash Standard*). U.S. Department of Commerce, Aug. 2015. DOI: 10.6028/NIST.FIPS.180-4. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf (cit. on p. 1).

[4] National Institute of Standards and Technology. *Cryptographic Algorithm Validation Program: Secure Hashing Test Vectors (byte-oriented and bit-oriented)*. CAVP, SHAVS. The bit-oriented set is the only authoritative external reference for hashing messages whose length is not a multiple of eight. See §4. 2026. URL: https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/secure-hashing (cit. on p. 5).

[5] Alex Shinn, John Cowan, and Arthur A. Gleckler. *Revised^7 Report on the Algorithmic Language Scheme*. Local copy: `refs/r7rs.pdf`, retrieved 2026-08-09 from small.r7rs.org. Cited here for the absence of any bitwise operation in the R7RS-small base library. 2013. URL: https://small.r7rs.org/attachment/r7rs.pdf (cit. on p. 3).